

CodeGraph - Feature Specification (Version 2)

Each feature includes a short description explaining its purpose and the information required from the parser/graph engine.

A. Repository Overview

Repository Statistics

Overall metrics such as files, folders, symbols, imports and exports.

Language Statistics

Distribution of supported programming languages in the repository.

Folder/File Explorer

Navigate the repository hierarchy from folders down to files.

B. Symbol Explorer

File Details

Display metadata and symbols defined inside a selected file.

Function Details

Inspect a function's signature, location and relationships.

Class Details

View class methods, fields, inheritance and implementations.

Interface Details

Inspect interface members and inheritance.

Variable Details

Display variable scope, type and declaration information.

C. Relationship Features

Calls

Functions directly invoked by the current function.

Called By

Functions that invoke the current function.

Imports

External modules or files imported by a file.

Exports

Symbols exposed by a file for other modules.

File Dependencies

Files that depend on one another through imports.

Module Dependencies

Relationships between higher-level project modules/folders.

Class Inheritance

Parent-child relationships between classes.

Interface Implementation

Classes implementing specific interfaces.

Object Instantiation

Places where objects are created using 'new'.

D. Graph Features

Call Graph

Interactive graph of function call relationships.

Dependency Graph

Visual representation of file dependencies.

Module Graph

Architecture view showing module-level connections.

Import Graph

Graph showing import/export relationships.

E. Search Features

Symbol Search

Locate functions, classes, interfaces or variables.

File Search

Quickly locate repository files.

Path Search

Search folders and directory hierarchy.

F. Analysis Features

Circular Dependency Detection

Detect dependency cycles between files or modules.

Dead Code Detection

Identify unreachable or unused code.

Unused Exports

Find exported symbols never imported elsewhere.

Unused Functions

Find functions with no incoming callers.

Reachability Analysis

Determine execution paths to a symbol.

Dependency Count

Count incoming and outgoing dependencies.

Fan-In

Number of incoming relationships to a symbol.

Fan-Out

Number of outgoing relationships from a symbol.

Most Connected Symbols

Rank symbols by graph connectivity.

Largest Files

Identify files with the greatest size or LOC.

Largest Classes

Rank classes by methods or members.

G. Impact Analysis

Change Impact

Estimate what changes are affected by a modification.

Delete Impact

Identify components broken if a symbol is removed.

Dependency Chain

Show dependency path between two entities.

Call Path

Display complete function invocation path.

H. Dashboard

Repository Health

Overall structural health score.

Complexity Ranking

Rank files/classes by complexity.

Circular Dependency Count

Summarize detected dependency cycles.

Largest Modules

Identify largest project modules.

Core Entities

Repository

Top-level project containing folders and files.

Symbols

Named program elements: functions, classes, interfaces, variables, enums, methods.

Relationships

Edges such as CALLS, IMPORTS, EXPORTS, EXTENDS, IMPLEMENTS, INSTANTIATES, DECLARES and CONTAINS.

Metadata

Supporting information like locations, modifiers, visibility, comments and complexity.

Pipeline Separation

Repository Walker

Discovers repository structure and files.

Parser

Produces raw ASTs from supported source files.

Symbol Extractor

Converts ASTs into language-independent symbols.

Relationship Extractor

Creates graph edges between symbols.

Graph Builder

Constructs and stores the repository graph.

Analysis Engine

Runs graph algorithms and repository analyses.